

实验 2 标准 SQL 语言和简单查询

班级: 07112304

学号: 1120233329

姓名: 陈墨霏

一、实验目的

掌握 SQL 程序设计基本规范、熟练运用 SQL 语言实现数据基本查询,包括单表、分组统计查询、连接查询等。

二、实验内容

- 导入实验数据集
- 用 SQL 语句实现以下查询:
 - 1) 查询所有供应商的名称、地址、联系电话。
 - 2) 查询 2014 年 1 10 月间的总价大于 1000 元的订单的编号、顾客编号等订单的所有信息。
 - 3) 统计每个顾客的订购金额。
 - 4) 查询订单平均金额超过 1000 元的顾客编号及其姓名。
 - 5) 查询与“金仓集团”在同一个国家的供应商编号、名称和地址信息。
 - 6) 查询供应价格大于零售价格的零件名、制造商名、零售价格和供应价格。
 - 7) 查询顾客“阿波罗”订购的订单编号、总价及其订购的零件编号、数量和零售价格。

注: 查询结果截图粘贴到实验报告中。

三、实验步骤

3.1 导入实验数据集

3.1.1 数据集说明

由于实验没有提供统一的数据集,因此我使用 Qwen 2.5 Max AI 生成了一些数据,并以 .csv 格式保存。每个表格的第一行是表头,每张表包含 10 条或 11 条数据。以下是 supplier.csv 的内容,作为示例。

```
supplier_id,supplier_name,address,nation_id,phone
1,红魔馆后勤部,幻想乡红魔乡,1,001-12345678
2,魔理沙的魔法商店,魔法之森深处,2,002-87654321
3,河城荷取工业,迷途竹林入口,3,003-11223344
4,妖梦的刀剑铺,冥界边境,4,004-55667788
5,咲夜管家服务,天界南门,5,005-99887766
6,地灵殿物资处,旧地狱核心区,6,006-13579246
7,风祝的神社商店,博丽神社旁,8,007-24681357
8,铃仙的月面物流,月之都环形山,10,008-36912587
9,八云紫的隙间贸易,妖怪之山山脚,7,009-15935748
10,火焰猫燐的怨灵回收站,旧地狱怨灵聚集地,6,010-75395168
11,金仓集团,幻想乡仓库区,1,011-88888888
```

3.1.2 数据集导入

由于实验中是首先建立好了表，然后再导入数据，因此在导入数据时需要注意先后顺序，以满足表间的外键约束（参照完整性约束）关系。

在实验一中，已经得到过以下参照关系图：

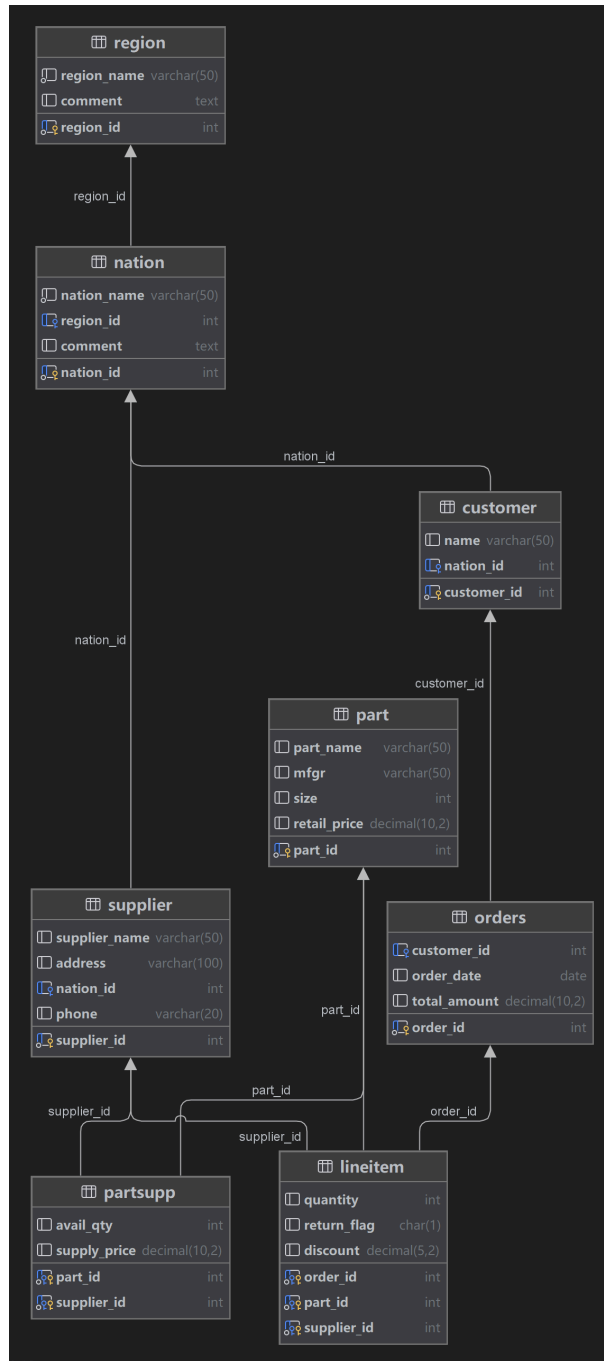


Figure 1: 表数据关系图

将关系图进行拓扑排序，得到的其中一种导入数据的顺序如下：

Region → Nation → Supplier → Customer → Part → PartSupp → Orders → Lineitem

于是按照以上顺序进行数据导入。

方式一：命令行输入 SQL 语句导入

例如，导入 `region.csv` 数据时，使用以下命令：

```
LOAD DATA INFILE '../lab2/data/region.csv'
INTO TABLE Supplier
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS -- 忽略表头
(supplier_id, supplier_name, address, nation_id, phone);
```

注意到报错：

```
The MySQL server is running with the --secure-file-priv option so it cannot
execute this statement
```

这是因为 MySQL 的安全选项限制了文件的读写权限。解决方法是将数据文件放在 MySQL 的默认目录下，或者在 MySQL 配置文件中修改 `secure-file-priv` 的值。

使用以下命令查看当前的 `secure-file-priv` 设置：

```
SHOW VARIABLES LIKE 'secure_file_priv';
```

之后将数据文件放在 MySQL 的默认目录下，或者在 MySQL 配置文件中修改 `secure-file-priv` 的值。然后重新执行导入命令即可。

方式二：DataGrip 图形化界面导入

SQL 语句的导入方式受到 MySQL 安全选项的限制，导入时还需要注意文件路径的问题，很不方便。而 DataGrip 不依赖 `LOAD DATA INFILE`，因此不受 `secure_file_priv` 的限制，可以从任意路径导入数据。

例如，导入 `region.csv` 数据时，使用以下步骤：

在 DataGrip 中，右键点击 `region` 表，选择 **导入/导出** — **从文件导入数据**。

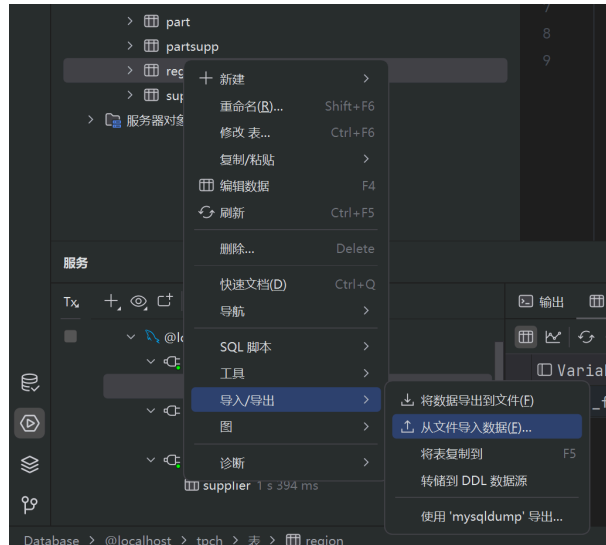


Figure 2: 导入数据

选择合适路径，点击确定。弹出导入窗口后，按照默认设置，点击确定，即可完成导入。

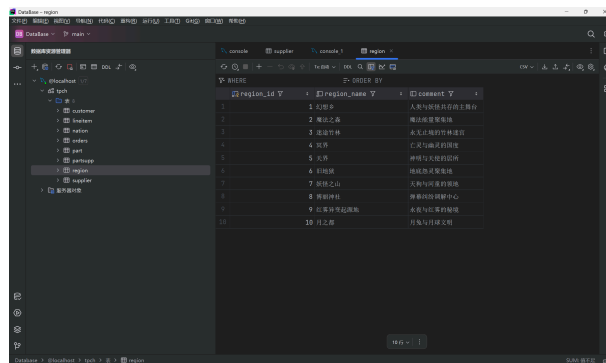


Figure 3: 导入成功

其余数据通过同样方式按照顺序依次导入即可。

3.2 查询数据

发起查询请求前，注意先选择架构：

```
USE tpch;
```

3.2.1 查询请求一

查询所有供应商的名称、地址、联系电话。

```
SELECT supplier_name, address, phone
FROM supplier;
```

解释:

- **SELECT** 语句用于选择要查询的列, **supplier_name**、**address** 和 **phone** 是要查询的列名。
- **FROM** 语句用于指定要查询的表, **supplier** 是要查询的表名。

3. 2. 2 查询请求二

查询 2014 年 1~10 月间的总价大于 1000 元的订单的编号、顾客编号等订单的所有信息。

```
SELECT *  
FROM orders  
WHERE order_date BETWEEN '2014-01-01' AND '2014-10-31'  
      AND total_amount > 1000;
```

解释:

- **SELECT *** 语句用于选择所有列, ***** 表示选择所有列。
- **FROM** 语句用于指定要查询的表, **Orders** 是要查询的表名。
- **WHERE** 语句用于指定查询条件,
order_date BETWEEN '2014-01-01' AND '2014-10-31' 表示查询日期在 2014 年 1 月 1 日至 2014 年 10 月 31 日之间的订单, **total_price > 1000** 表示查询总价大于 1000 元的订单。

3. 2. 3 查询请求三

统计每个顾客的订购金额。

```
SELECT customer_id, SUM(total_amount) AS total_amount_sum  
FROM orders  
GROUP BY customer_id;
```

解释:

- **SELECT** 语句用于选择要查询的列, **customer_id** 是顾客编号, **SUM(total_amount)** 用于计算每个顾客的订购金额总和, 并将其命名为 **total_amount_sum**。
- **FROM** 语句用于指定要查询的表, **orders** 是要查询的表名。
- **GROUP BY** 语句用于将结果按顾客编号分组, **customer_id** 是要分组的列名。

3. 2. 4 查询请求四

查询订单平均金额超过 1000 元的顾客编号及其姓名。

```

SELECT customer_id, name
FROM customer
WHERE customer_id IN (
    SELECT customer_id
    FROM orders
    GROUP BY customer_id
    HAVING AVG(total_amount) > 1000
);

```

解释：

- **SELECT** 语句用于选择要查询的列，**customer_id** 是顾客编号，**name** 是顾客姓名。
- **FROM** 语句用于指定要查询的表，**customer** 是要查询的表名。
- **WHERE** 语句用于指定查询条件，**customer_id IN (...)** 表示查询顾客编号在子查询结果中的顾客。
- **SELECT customer_id FROM orders GROUP BY customer_id HAVING AVG(total_amount) > 1000** 用于计算每个顾客的平均订购金额，并筛选出平均金额超过 1000 元的顾客编号。

3.2.5 查询请求五

查询与“金仓集团”在同一个国家的供应商编号、名称和地址信息。

```

SELECT supplier_id, supplier_name, address
FROM supplier
WHERE nation_id = (
    SELECT nation_id
    FROM supplier
    WHERE supplier_name = '金仓集团'
);

```

解释：

- **SELECT** 语句用于选择要查询的列，**supplier_id** 是供应商编号，**supplier_name** 是供应商名称，**address** 是供应商地址。
- **FROM** 语句用于指定要查询的表，**supplier** 是要查询的表名。
- **WHERE** 语句用于指定查询条件，**nation_id = (...)** 表示查询与“金仓集团”在同一个国家的供应商。
- 子查询 **SELECT nation_id FROM supplier WHERE supplier_name = '金仓集团'** 用于获取“金仓集团”的国家编号。

3.2.6 查询请求六

查询供应价格大于零售价格的零件名、制造商名、零售价格和供应价格。

```
SELECT part_name, supplier_name, retail_price, supply_price
FROM part
JOIN partsupp ON part.part_id = partsupp.part_id
JOIN supplier ON partsupp.supplier_id = supplier.supplier_id
WHERE supply_price > retail_price;
```

解释：

- **SELECT** 语句用于选择要查询的列，**part_name** 是零件名称，**supplier_name** 是供应商名称，**retail_price** 是零售价格，**supply_price** 是供应价格。
- **FROM** 语句用于指定要查询的表，**part** 是要查询的零件表，**partsupp** 是零件供应表，**supplier** 是供应商表。
- **JOIN** 语句用于连接多个表，**ON** 语句用于指定连接条件，**part.part_id = partsupp.part_id** 和 **partsupp.supplier_id = supplier.supplier_id** 分别表示零件表和零件供应表的连接条件。
- **WHERE** 语句用于指定查询条件，**supply_price > retail_price** 表示查询供应价格大于零售价格的零件。

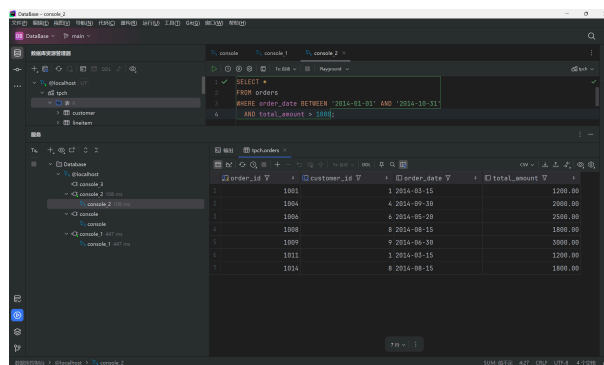
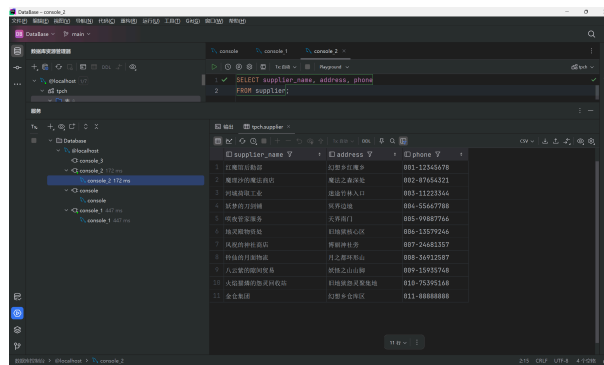
3.2.7 查询请求七

查询顾客“阿波罗”订购的订单编号、总价及其订购的零件编号、数量和零售价格。

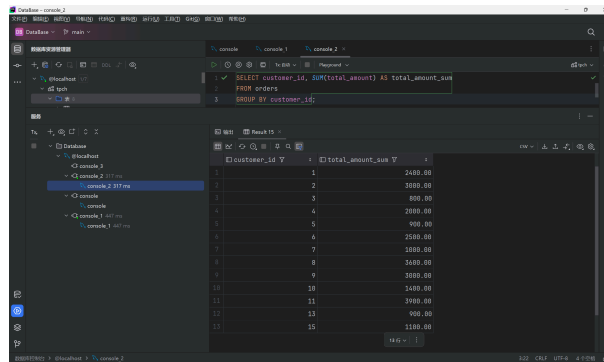
```
SELECT
    o.order_id,
    o.total_amount,
    l.part_id,
    l.quantity,
    p.retail_price
FROM
    Orders o
JOIN
    Customer c ON o.customer_id = c.customer_id
JOIN
    Lineitem l ON o.order_id = l.order_id
JOIN
    Part p ON l.part_id = p.part_id
WHERE
    c.name = '阿波罗';
```

解释：

- **SELECT** 语句用于选择要查询的列，**o.order_id** 是订单编号，**o.total_amount** 是订单总价，**l.part_id** 是零件编号，**l.quantity** 是订购数量，**p.retail_price** 是零售价格。
- **FROM** 语句用于指定要查询的表，**Orders** 是订单表，**Customer** 是顾客表，**Lineitem** 是订单明细表，**Part** 是零件表。



4.1.3 查询结果三



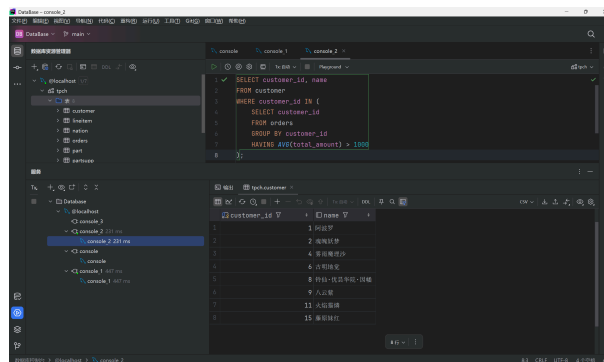
customer_id	total_amount_sum
1	2400.00
2	2000.00
3	800.00
4	2000.00
5	900.00
6	2000.00
7	1800.00
8	1600.00
9	2000.00
10	1400.00
11	1800.00
12	900.00
13	1100.00
14	1100.00
15	1100.00

Figure 6: 统计每个顾客的订购金额

分析:

- 查询结果显示了每个顾客的订购金额总和。
- 使用了 **SUM** 函数对金额进行汇总，并结合 **GROUP BY** 按顾客编号分组。
- 结果表明，SQL 聚合函数和分组操作能够有效地统计数据。

4.1.4 查询结果四



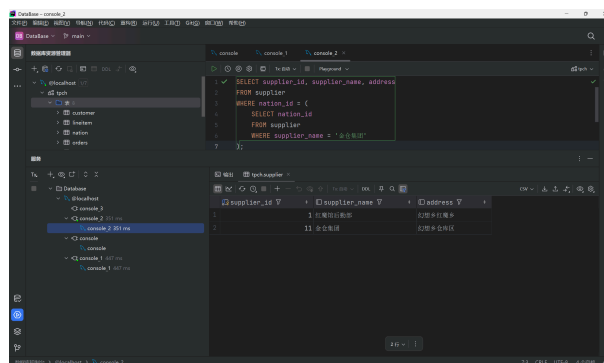
customer_id	name
1	何启祥
2	陈伟斌
4	蔡国耀伟达
5	何伟强
8	陈伟达陈伟达: 陈伟
9	王达智
11	宋伟强
15	陈伟强

Figure 7: 查询订单平均金额超过 1000 元的顾客编号及其姓名

分析:

- 查询结果显示了符合条件的顾客编号及其姓名。
- 使用了子查询和 **HAVING** 子句，筛选出平均金额超过 1000 元的顾客。
- **HAVING** 子句的作用是对分组后的数据进行过滤，与 **WHERE** 的作用不同。

4.1.5 查询结果五



supplier_id	supplier_name	address
1	金仓集团	金仓集团总部
11	金仓集团	金仓集团分公司

Figure 8: 查询与“金仓集团”在同一个国家的供应商编号、名称和地址信息

分析:

- 查询结果显示了与“金仓集团”在同一个国家的供应商信息。
- 使用了子查询来获取“金仓集团”的国家编号，并通过 **WHERE** 条件进行匹配。
- 子查询的嵌套使用提高了查询的灵活性。

4.1.6 查询结果六

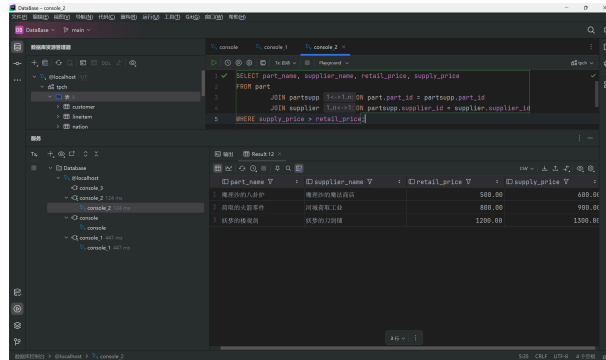


Figure 9: 查询供应价格大于零售价格的零件名、制造商名、零售价格和供应价格

分析:

- 查询结果显示了符合条件的零件及其相关信息。
- 使用了多表连接 (**JOIN**) 来关联零件、供应商和供应价格表。
- **supply_price > retail_price** 条件确保了筛选出供应价格大于零售价格的记录。

4.1.7 查询结果七

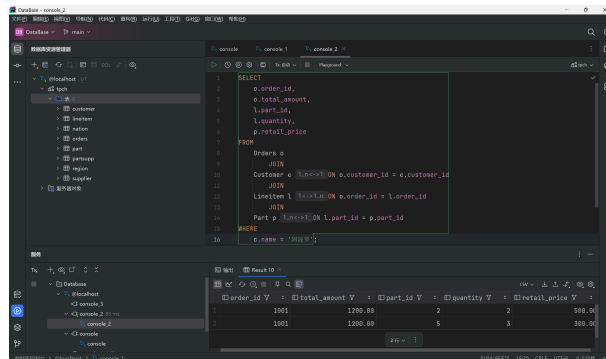


Figure 10: 查询顾客“阿波罗”订购的订单编号、总价及其订购的零件编号、数量和零售价格

分析:

- 查询结果显示了顾客“阿波罗”订购的订单及其详细信息。
- 使用了多表连接 (**JOIN**) 来关联订单、顾客、订单明细和零件表。
- **WHERE** 条件确保了仅查询顾客名称为“阿波罗”的记录。
- 查询展示了复杂查询中多表关联的强大功能。

4.2 实验总结

本次实验完成了 SQL 语言的基本操作，包括单表查询、分组统计查询、连接查询等任务。通过导入实验数据集并执行多种查询操作，验证了 SQL 语句的正确性和查询结果

的准确性。同时，实验中还解决了数据导入过程中遇到的权限问题，熟悉了不同工具的使用方法。

五、实验收获与体会

通过本次实验，我对 SQL 语言的基本语法和查询操作有了更深入的理解，掌握了如何使用 SQL 进行数据查询和分析。同时，我也认识到在实际应用中，数据的导入和处理是一个重要的环节，需要注意数据的完整性和一致性。通过实验，我提高了自己的 SQL 编程能力，为后续的数据库学习打下了基础。

附录：程序清单及说明

以下是本实验中涉及的主要程序文件及其说明：

- 1) **search.sql**: 包含了所有的查询请求和对应的 SQL 语句。

以上文件均已在实验中详细说明，并附有必要的注释以便理解和复现实验过程。